

Техническое задание на разработку прототипа веб-интерфейса управления жизненным циклом рабочего места программиста

Версия 1.0

Документ предназначен для проектирования и реализации минимально жизнеспособного, но защищаемого программного продукта в рамках ВКР и преддипломной практики.

Параметр	Значение
Тема ВКР	Интерфейс управления жизненным циклом рабочего места программиста
Класс системы	Локальное/веб-приложение для управления проектными артефактами и связями между ними
Основной акцент	Онтологическая модель предметной области, граф связей и базовый анализ влияния изменений
Целевой результат	Рабочий прототип, который можно показать, протестировать и описать в отчете
Ограничение масштаба	Не является полноценной IDE, Jira, GitHub, Protégé или корпоративной ALM-системой
Дата подготовки	2026

Содержание документа

- 1. Общие сведения
- 2. Назначение и цели системы
- 3. Термины, сокращения и соглашения
- 4. Границы системы и ограничения MVP
- 5. Роли пользователей и сценарии использования
- 6. Концептуальная модель системы
- 7. Функциональные требования
- 8. Требования к онтологической модели и графу связей
- 9. Требования к данным и базе данных
- 10. Требования к пользовательскому интерфейсу
- 11. Требования к API
- 12. Алгоритмы и бизнес-правила
- 13. Нефункциональные требования
- 14. Требования к стеку технологий и архитектуре
- 15. Развертывание и эксплуатация
- 16. Тестирование и приемка
- 17. Демонстрационный сценарий
- 18. План реализации
- 19. Риски и решения
- Приложения

1. Общие сведения

Разрабатываемая система представляет собой прототип веб-интерфейса для ведения, связывания и просмотра основных артефактов разработки программного обеспечения. Под рабочим местом программиста в рамках данной работы понимается не физическое рабочее место и не полноценная среда разработки, а информационное пространство, в котором хранятся требования, сущности предметной области, проектные элементы, фрагменты программного кода, тесты, документы и связи между ними.

Центральная идея системы: каждый значимый элемент проекта рассматривается как артефакт, а связи между артефактами образуют прикладную онтологическую модель. Благодаря этому пользователь может не только хранить элементы проекта, но и видеть, какие требования, сущности, модули, файлы, функции и тесты потенциально затрагиваются при изменениях.

Система должна быть достаточно простой для реализации одним разработчиком в рамках диплома, но достаточно содержательной, чтобы продемонстрировать анализ предметной области, работу с требованиями, проектирование архитектуры, разработку интерфейса, хранение данных, визуализацию графа и базовые механизмы анализа влияния изменений.

1.1. Основания для разработки

- тема выпускной квалификационной работы: «Интерфейс управления жизненным циклом рабочего места программиста»;
- необходимость подготовить результат преддипломной практики: анализ предметной области, постановку задачи, выбор стека, архитектуру и частичный/рабочий прототип;

- рекомендации научного руководителя использовать онтологии как основу связывания элементов предметной области и проектных артефактов;
- практическая потребность отслеживать последствия изменения требований, описания предметной области и проектных решений.

1.2. Краткая формулировка продукта

Продукт: прототип веб-приложения, позволяющий создавать проект, вести реестр артефактов, описывать предметную область, фиксировать требования, добавлять элементы реализации, связывать их между собой, отображать связи в виде графа, выполнять базовый анализ влияния изменений и экспортировать модель в машиночитаемый формат.

1.3. Принципиальная позиция по масштабу

Важное ограничение: система не должна пытаться заменить промышленную IDE, систему управления требованиями, task-трекер, Git-хостинг или профессиональный редактор онтологий. В дипломе защищается именно интерфейс управления связанными артефактами, а не универсальная платформа разработки. Код в системе является одним из артефактов жизненного цикла, наряду с требованиями, сущностями предметной области, архитектурными элементами, тестами и документацией.

2. Назначение и цели системы

Назначение системы — поддержка работы программиста или студента-разработчика с проектными артефактами в условиях изменяющихся требований и уточняющейся предметной области. Система помогает фиксировать знания о проекте, связывать их между собой и получать наглядное представление о том, какие элементы проекта зависят друг от друга.

2.1. Проблема, которую решает система

В процессе разработки программного обеспечения требования, представления о предметной области и проектные решения постоянно меняются. Если требования, сущности предметной области, архитектурные решения, код и тесты хранятся разрозненно, разработчику сложно понять, какие элементы проекта нужно проверить или изменить после правки одного требования. Это приводит к рассогласованию документации, архитектуры, тестов и реализации.

2.2. Цель разработки

Цель разработки — создать рабочий прототип программного интерфейса для управления связанными артефактами рабочего места программиста на основе онтологической модели предметной области и графа связей.

2.3. Практические задачи системы

1. Хранить проекты и их описания.
2. Хранить сущности предметной области и их свойства.
3. Хранить функциональные и нефункциональные требования.
4. Хранить проектные элементы: модули, компоненты, файлы, функции, тесты, документы, дефекты и запросы на изменение.

5. Позволять пользователю создавать типизированные связи между артефактами.
6. Визуализировать модель проекта в виде интерактивного графа.
7. Определять потенциально затронутые элементы при изменении выбранного требования или сущности.
8. Предоставлять простой редактор/просмотрщик фрагментов кода.
9. Экспортировать модель проекта в JSON и RDF/Turtle для демонстрации совместимости с семантическими технологиями.
10. Предоставлять демонстрационные данные для защиты и тестирования.

3. Термины, сокращения и соглашения

Термин	Определение
Артефакт	Любой значимый элемент проекта: требование, сущность предметной области, модуль, файл, функция, тест, документ, дефект, решение и т.д.
Онтологическая модель	Формализованное представление понятий предметной области и связей между ними. В данном проекте реализуется прикладная онтологическая модель, хранящаяся в БД и экспортируемая в RDF/Turtle.
Граф связей	Набор узлов-артефактов и ребер-связей между ними. Используется для визуальной навигации и анализа влияния изменений.
Трассируемость	Возможность проследить связь между требованием, предметной областью, проектным решением, кодом и тестом.
Анализ влияния изменений	Процедура поиска элементов проекта, которые потенциально затрагиваются при изменении выбранного артефакта.
MVP	Минимально жизнеспособная версия продукта, достаточная для демонстрации основных идей работы.
CRUD	Create, Read, Update, Delete — базовые операции создания, просмотра, редактирования и удаления записей.
RDF	Resource Description Framework — модель представления данных в виде троек «субъект — предикат — объект».
OWL	Web Ontology Language — язык описания онтологий. В MVP полная поддержка OWL-вывода не требуется.
API	Программный интерфейс взаимодействия frontend и backend.

3.1. Соглашения по идентификаторам

- Функциональные требования обозначаются как FR-001, FR-002 и т.д.
- Нефункциональные требования обозначаются как NFR-001, NFR-002 и т.д.
- Тестовые сценарии обозначаются как TC-001, TC-002 и т.д.
- Артефакты внутри проекта могут иметь человекочитаемые коды: REQ-001, ENT-001, MOD-001, FILE-001, FUNC-001, TEST-001.
- Связи обозначаются системным кодом типа связи: part_of, depends_on, realizes, verifies, impacts и т.д.

4. Границы системы и ограничения MVP

Чтобы проект был реализуемым в срок, система строится как локальный однопользовательский прототип. Это не недостаток, а сознательное ограничение области разработки. В дипломе можно показать, что выбранная архитектура допускает дальнейшее расширение, но первая версия должна быть простой и рабочей.

4.1. Входит в MVP

- создание и редактирование проектов;
- создание, редактирование, удаление и просмотр артефактов;
- типизация артефактов по заранее заданному справочнику;
- создание и удаление связей между артефактами;
- проверка базовых ограничений связей;
- визуализация графа проекта;
- простой редактор фрагментов кода с подсветкой синтаксиса;
- анализ влияния изменений на основе обхода графа;
- поиск и фильтрация артефактов;
- экспорт проекта в JSON;
- экспорт онтологической модели в RDF/Turtle;
- загрузка демонстрационного проекта.

4.2. Не входит в MVP

- компиляция, запуск и отладка кода;
- полноценная интеграция с Git;
- автоматический парсинг исходного кода и построение AST;
- многопользовательская работа, роли доступа и сложная авторизация;
- полноценный OWL reasoner и SPARQL endpoint;
- совместное редактирование в реальном времени;
- замена Jira, YouTrack, GitHub, VS Code, IntelliJ IDEA, Enterprise Architect или Protégé;
- автоматическое извлечение требований из текстов;
- промышленная система уведомлений и workflow согласования.

5. Роли пользователей и сценарии использования

Для MVP достаточно одной основной роли — пользователь-разработчик.

Дополнительные роли можно описать как перспективу развития, но не реализовывать.

Роль	Статус	Действия
Разработчик/аналитик	Основная роль MVP	Создает проект, описывает предметную область, требования, артефакты реализации, связи, запускает анализ влияния.
Научный руководитель/проверяющий	Наблюдательная роль, без отдельной авторизации	Просматривает демонстрационный проект, граф связей и результаты анализа влияния.
Администратор проекта	Перспектива развития	Управляет пользователями, правами, шаблонами типов и настройками проекта. В MVP не реализуется как отдельная роль.

5.1. Основные пользовательские сценарии

ID	Сценарий	Описание
UC-001	Создание проекта	Пользователь задает название и описание проекта. Система создает пустой проект со стандартными типами артефактов и связей.
UC-002	Описание предметной области	Пользователь добавляет сущности предметной области, например «Пользователь», «Задача», «Комментарий».
UC-003	Фиксация требования	Пользователь добавляет требование, указывает тип, статус, приоритет и описание.
UC-004	Связывание требования с реализацией	Пользователь связывает требование с сущностью предметной области, модулем, файлом, функцией и тестом.
UC-005	Просмотр графа	Пользователь открывает граф проекта, фильтрует узлы по типам и выбирает интересующий артефакт.
UC-006	Анализ влияния изменений	Пользователь выбирает измененное требование. Система показывает потенциально затронутые сущности, модули, файлы, функции, тесты и документы.
UC-007	Экспорт модели	Пользователь экспортирует проект в JSON или RDF/Turtle для отчета, резервного копирования или демонстрации.

6. Концептуальная модель системы

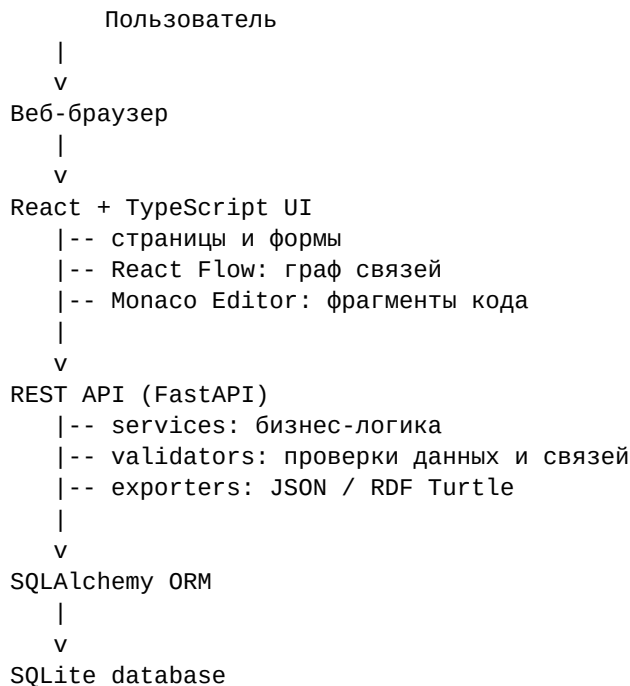
Система строится вокруг универсального понятия «артефакт». Это снижает сложность реализации: вместо отдельных таблиц и экранов для каждого типа можно использовать единый механизм хранения, фильтрации, связывания и отображения. Специализация достигается через тип артефакта и дополнительные поля `metadata_json`.

6.1. Основные сущности

Сущность	Назначение
Project	Проект, внутри которого создаются все артефакты и связи.
ArtifactType	Справочник типов артефактов: Requirement, DomainConcept, Module, File, Function, TestCase и т.д.
Artifact	Универсальная запись проектного элемента с кодом, названием, описанием, статусом, приоритетом и расширяемыми метаданными.
RelationType	Справочник типов связей: part_of, depends_on, realizes, verifies, impacts и т.д.
ArtifactRelation	Конкретная направленная или ненаправленная связь между двумя артефактами.

Сущность	Назначение
CodeFragment	Фрагмент кода, привязанный к артефакту типа File, Function, Module или Component.
ChangeLog	Журнал изменений сущностей системы. Нужен для прозрачности демонстрации и отладки.
ExportRecord	Запись о сформированном экспорте модели проекта.

6.2. Архитектурная схема верхнего уровня



6.3. Логика представления артефактов

Все элементы проекта должны отображаться единообразно: у каждого артефакта есть системный идентификатор, человекочитаемый код, тип, название, описание, статус, приоритет, набор метаданных, дата создания и дата обновления. Для специализированных данных используется поле `metadata_json`. Например, для требования там можно хранить `rationale`, `acceptance_criteria`, `source`; для теста — `test_type`, `preconditions`, `expected_result`; для файла — `path`, `language`.

7. Функциональные требования

Все функциональные требования обязательны для рабочей версии MVP, кроме требований с приоритетом **SHOULD**. При необходимости можно реализовать **SHOULD** после базовой демонстрационной версии.

ID	Название	Приоритет	Требование
FR-001	Управление проектами	MUST	Система должна позволять создавать, просматривать, редактировать и удалять проекты.

ID	Название	Приоритет	Требование
FR-002	Панель проекта	MUST	Система должна показывать краткую статистику проекта: количество артефактов, связей, требований, тестов и элементов без связей.
FR-003	Справочник типов артефактов	MUST	Система должна иметь преднастроенный набор типов артефактов и позволять использовать их при создании записей.
FR-004	CRUD артефактов	MUST	Система должна позволять создавать, просматривать, редактировать и удалять артефакты любого типа.
FR-005	Управление требованиями	MUST	Система должна поддерживать требования как специализированный тип артефакта с типом, статусом, приоритетом и критериями приемки.
FR-006	Управление сущностями предметной области	MUST	Система должна позволять описывать понятия предметной области и связывать их с требованиями и реализацией.
FR-007	Управление элементами реализации	MUST	Система должна поддерживать модули, компоненты, файлы, функции и тесты как артефакты реализации.
FR-008	Редактор фрагментов кода	MUST	Система должна предоставлять простой редактор/просмотрщик кода, привязанный к выбранному артефакту.
FR-009	Создание связей	MUST	Система должна позволять создавать типизированные связи между двумя артефактами.
FR-010	Проверка связей	MUST	Система должна проверять недопустимые связи: пустые ссылки, связь с самим собой, запрещенные типы источника/цели, циклы для part_of и is_a.
FR-011	Визуализация графа	MUST	Система должна отображать артефакты и связи проекта в виде интерактивного графа.
FR-012	Фильтрация графа	SHOULD	Система должна позволять фильтровать граф по типам артефактов и типам связей.
FR-013	Анализ влияния изменений	MUST	Система должна строить список потенциально затронутых артефактов для выбранного узла.

ID	Название	Приоритет	Требование
FR-014	Поиск артефактов	MUST	Система должна поддерживать поиск по коду, названию и описанию артефактов.
FR-015	Журнал изменений	SHOULD	Система должна фиксировать основные изменения артефактов и связей в журнале.
FR-016	Экспорт JSON	MUST	Система должна экспортировать проект со всеми артефактами и связями в JSON.
FR-017	Экспорт RDF/Turtle	MUST	Система должна экспортировать онтологическую модель проекта в RDF/Turtle.
FR-018	Импорт JSON	SHOULD	Система должна позволять импортировать ранее экспортированный JSON проекта.
FR-019	Демонстрационный проект	MUST	Система должна иметь команду или кнопку загрузки демонстрационного проекта.
FR-020	Диагностика неполноты	SHOULD	Система должна показывать артефакты без связей и требования без тестов.

7.1. Детализация ключевых требований

FR-004. CRUD артефактов

- При создании артефакта пользователь обязан указать тип, код и название.
- Описание, статус, приоритет и метаданные могут быть заполнены позднее.
- Код артефакта должен быть уникален внутри проекта.
- Удаление артефакта должно удалять или блокировать связанные отношения. Для MVP допустимо каскадное удаление связей с подтверждением пользователя.
- После сохранения артефакт должен появиться в реестре, графе и результатах поиска.

FR-009. Создание связей

- Связь создается между source_artifact и target_artifact.
- Для направленных связей направление должно быть явно отображено в интерфейсе.
- Пользователь выбирает тип связи из справочника RelationType.
- Если тип связи ограничивает допустимые типы источника и цели, система должна проверить это перед сохранением.
- Для каждой связи можно указать краткий комментарий, объясняющий смысл связи.

FR-013. Анализ влияния изменений

- Пользователь выбирает один исходный артефакт и глубину обхода графа.
- Система должна показать список найденных артефактов, сгруппированный по типам.
- Для каждого результата система должна показать путь связи от исходного артефакта до найденного элемента.
- Результаты должны быть доступны в виде списка и подсвечены на графе.

- Минимальный алгоритм — обход графа в ширину по связям, влияющим на трассируемость.

8. Требования к онтологической модели и графу связей

Онтологическая составляющая в MVP реализуется прикладно: через типы артефактов, типы связей, ограничения связей и экспорт в RDF/Turtle. Это дает дипломную связь с онтологиями, но не перегружает реализацию полноценной OWL-инфраструктурой.

8.1. Базовые классы онтологической модели

Класс	Смысл
Project	Проект разработки ПО
DomainConcept	Понятие предметной области: объект, процесс, роль, документ, событие, правило.
Requirement	Требование к системе: функциональное, нефункциональное, ограничение, бизнес-правило.
Module	Логический модуль системы.
Component	Компонент внутри модуля или отдельная часть архитектуры.
File	Файл реализации или конфигурации.
Function	Функция, метод, endpoint или процедура.
TestCase	Тестовый случай или проверка требования.
Document	Документ или раздел документации.
Defect	Ошибка, проблема или несоответствие.
ChangeRequest	Запрос на изменение требования, модели или реализации.
Decision	Проектное решение с аргументацией.

8.2. Типы связей

Код	Название	Допустимые типы	Направл.	Комментарий
is_a	Род-вид / специализация	DomainConcept -> DomainConcept	Да	Для таксономии понятий. Циклы запрещены.
part_of	Часть-целое	Любой Любой ->	Да	Для структуры системы, модуля, документа. Циклы запрещены.
depends_on	Зависит от	Любой Любой ->	Да	Для технических и логических зависимостей. Учитывается в анализе влияния.
realizes	Реализует	Module/ Component/Function/File -> Requirement	Да	Связь реализации с требованием.
implemented_in	Реализовано в	Requirement/ Function -> File/Module	Да	Альтернативная удобная связь от требования к реализации.
describes	Описывает	Document -> Artifact	Да	Связь документации с объектом описания.
verifies	Проверяет	TestCase -> Requirement/Function	Да	Связь теста с проверяемым элементом.

Код	Название	Допустимые	Направл.	Комментарий
		типы		
affects	Влияет на	Любой -> Любой	Да	Явная связь влияния, всегда участвует в анализе.
conflicts_with	Конфликтует с	Requirement -> Requirement	Нет	Фиксация противоречия требований или решений.
related_to	Связан с	Любой -> Любой	Нет	Слабая ассоциативная связь, по умолчанию не используется для строгого анализа.
refines	Уточняет	Requirement -> Requirement	Да	Декомпозиция требований.
uses	Использует	Function/ Module -> Function/Module	Да	Техническое использование компонента.

8.3. Правила онтологической модели

1. Каждый узел графа является артефактом и принадлежит ровно одному проекту.
2. Каждая связь принадлежит ровно одному проекту и соединяет два существующих артефакта того же проекта.
3. Связи is_a и part_of не должны образовывать циклы.
4. Связь с самим собой запрещена для всех типов, кроме явно разрешенной служебной связи. В MVP такие связи не нужны.
5. Для каждого типа связи должны быть заданы человекочитаемое название, системный код, направленность и признак участия в анализе влияния.
6. Артефакт может иметь несколько исходящих и входящих связей.
7. В графе должны визуально различаться типы артефактов и направления связей.
8. Слабые связи related_to не должны автоматически трактоваться как обязательное влияние, если пользователь не включил их в настройках анализа.

8.4. Экспорт RDF/Turtle

Экспорт RDF/Turtle нужен как доказательство того, что внутренняя модель может быть представлена в семантическом формате. Полный OWL-вывод в MVP не требуется. Экспорт должен строить тройки по принципу: артефакт является ресурсом, тип артефакта является классом, связь является RDF-свойством, а поля артефакта являются data properties.

```
@prefix wp: <urn:workplace:> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

wp:REQ-001 rdf:type wp:Requirement ;
  rdfs:label "Пользователь должен авторизоваться" ;
  wp:status "approved" ;
  wp:priority "high" .

wp:ENT-001 rdf:type wp:DomainConcept ;
  rdfs:label "Пользователь" .

wp:MOD-001 rdf:type wp:Module ;
  rdfs:label "Auth" ;
  wp:realizes wp:REQ-001 ;
```

```
wp:depends_on wp:ENT-001 .
```

```
wp:TEST-001 rdf:type wp:TestCase ;  
  rdfs:label "Проверка авторизации" ;  
  wp:verifies wp:REQ-001 .
```

9. Требования к данным и базе данных

В MVP рекомендуется использовать SQLite. Для дипломного прототипа это достаточно: не нужно настраивать сервер БД, проще развернуть и продемонстрировать результат. При этом доступ к данным должен быть реализован через SQLAlchemy ORM, чтобы при необходимости можно было заменить SQLite на PostgreSQL.

9.1. Минимальная схема БД

Таблица	Ключевые поля	Назначение
projects	id, name, description, created_at, updated_at	Проекты пользователя.
artifact_types	id, code, name, category, description, icon, color	Справочник типов артефактов. Заполняется стартовыми значениями.
artifacts	id, project_id, type_id, code, title, description, status, priority, metadata_json, created_at, updated_at	Универсальные артефакты проекта.
relation_types	id, code, name, description, directed, affects_impact, source_types_json, target_types_json, color	Справочник типов связей и их ограничений.
artifact_relations	id, project_id, source_artifact_id, target_artifact_id, relation_type_id, label, weight, comment, created_at	Связи между артефактами.
code_fragments	id, artifact_id, language, file_path, content, start_line, end_line, created_at, updated_at	Фрагменты кода, привязанные к артефактам.
change_log	id, project_id, entity_type, entity_id, operation, old_value_json, new_value_json, created_at	Журнал изменений.
export_records	id, project_id, format, file_name, created_at	История экспортов.

9.2. Статусы и справочники

Справочник	Значения
Статус требования	draft, proposed, approved, implemented, tested, deprecated
Статус артефакта	draft, active, changed, deprecated
Приоритет	low, medium, high, critical
Тип требования	functional, non_functional, constraint, business_rule
Тип теста	unit, integration, ui, acceptance, manual
Язык фрагмента кода	python, typescript, javascript, sql, html, css, text

9.3. Правила целостности данных

1. projects.name не должен быть пустым.
2. artifacts.code должен быть уникален в пределах project_id.
3. artifacts.title не должен быть пустым.

4. artifact_relations.source_artifact_id и target_artifact_id должны ссылаться на существующие артефакты того же project_id.
5. При удалении артефакта все связи с ним должны быть удалены каскадно или пользователь должен получить предупреждение. Для MVP допускается каскадное удаление после подтверждения.
6. metadata_json должен быть валидным JSON-объектом.
7. Для связей part_of и is_a перед сохранением выполняется проверка на цикл.
8. relation_types.code и artifact_types.code должны быть уникальными.

10. Требования к пользовательскому интерфейсу

Интерфейс должен быть простым, без избыточного визуального шума. Главный критерий — пользователь должен быстро понять, где создать артефакт, где связать его с другими элементами, где посмотреть граф и где запустить анализ влияния.

10.1. Общая компоновка

- верхняя панель: название приложения, текущий проект, кнопки импорта/экспорта;
- левая боковая навигация: Проекты, Панель проекта, Артефакты, Требования, Онтология/граф, Код, Анализ влияния, Настройки;
- центральная область: текущая таблица, форма, граф или редактор;
- правая контекстная панель: сведения о выбранном артефакте, входящие/исходящие связи, быстрые действия;
- модальные окна: создание/редактирование артефакта, создание связи, подтверждение удаления.

10.2. Экранные формы

Экран	Содержание
Проекты	Список проектов, создание проекта, открытие проекта, удаление проекта.
Панель проекта	Карточки статистики, последние изменения, артефакты без связей, требования без тестов.
Артефакты	Таблица всех артефактов с фильтрами по типу, статусу и приоритету.
Требования	Отдельное представление требований с типом, статусом, приоритетом и критериями приемки.
Онтология/граф	Интерактивный граф узлов и связей, фильтры, выделение выбранного артефакта.
Карточка артефакта	Название, тип, описание, метаданные, входящие и исходящие связи, кнопки редактирования.
Код	Редактор фрагмента кода, выбор языка, путь файла, привязка к артефакту.
Анализ влияния	Выбор исходного артефакта, глубина обхода, типы связей, список затронутых элементов и путь влияния.
Экспорт/импорт	Кнопки экспорта JSON, RDF/Turtle, импорта JSON, загрузки демонстрационных данных.
Настройки	Справочники типов артефактов и связей в режиме просмотра. Редактирование справочников можно не реализовывать в MVP.

10.3. Требования к графу

1. Граф должен строиться на основе artifacts и artifact_relations выбранного проекта.
2. Узлы должны иметь подпись code + title.
3. Типы артефактов должны визуально различаться цветом или иконкой.
4. Направленные связи должны отображаться стрелкой.
5. При клике на узел открывается карточка артефакта.
6. При клике на связь показывается тип связи и комментарий.
7. Должна быть возможность приблизить, отдалить и перемещать граф.
8. Должна быть кнопка «показать связанные» для фокусировки на выбранном узле.
9. Для больших графов допустимо показывать предупреждение и предлагать фильтры.

10.4. Требования к формам ввода

- обязательные поля должны быть помечены визуально;
- ошибки валидации должны показываться рядом с полем;
- сохранение должно быть недоступно при критических ошибках;
- после успешного сохранения пользователь должен получать понятное уведомление;
- удаление должно требовать подтверждения;
- длинные описания должны вводиться в многострочном поле.

11. Требования к API

API реализуется в стиле REST. Все данные передаются в JSON. Для MVP достаточно синхронных HTTP-запросов. WebSocket не требуется.

Метод	URL	Назначение
GET	/api/projects	Получить список проектов.
POST	/api/projects	Создать проект.
GET	/api/projects/{project_id}	Получить проект.
PATCH	/api/projects/{project_id}	Обновить проект.
DELETE	/api/projects/{project_id}	Удалить проект.
GET	/api/projects/{project_id}/artifacts	Получить артефакты проекта с фильтрами.
POST	/api/projects/{project_id}/artifacts	Создать артефакт.
GET	/api/artifacts/{artifact_id}	Получить артефакт.
PATCH	/api/artifacts/{artifact_id}	Обновить артефакт.
DELETE	/api/artifacts/{artifact_id}	Удалить артефакт.
GET	/api/projects/{project_id}/relations	Получить связи проекта.
POST	/api/projects/{project_id}/relations	Создать связь.
DELETE	/api/relations/{relation_id}	Удалить связь.
GET	/api/projects/{project_id}/graph	Получить данные графа: nodes и edges.
POST	/api/projects/{project_id}/impact	Запустить анализ влияния.
GET	/api/artifacts/{artifact_id}/code	Получить фрагменты кода артефакта.
POST	/api/artifacts/{artifact_id}/code	Создать/обновить фрагмент кода.
GET	/api/projects/{project_id}/export/json	Экспорт проекта в JSON.
GET	/api/projects/{project_id}/export/ turtle	Экспорт проекта в RDF/Turtle.
POST	/api/projects/import/json	Импорт проекта из JSON.

Метод	URL	Назначение
POST	/api/demo/load	Загрузить демонстрационный проект.

11.1. Пример JSON артефакта

```
{
  "id": "uuid",
  "project_id": "uuid",
  "type": "Requirement",
  "code": "REQ-001",
  "title": "Пользователь должен авторизоваться",
  "description": "Система должна позволять пользователю войти по логину и паролю.",
  "status": "approved",
  "priority": "high",
  "metadata": {
    "requirement_type": "functional",
    "acceptance_criteria": [
      "При корректных данных пользователь попадает в систему",
      "При некорректных данных отображается ошибка"
    ]
  },
  "created_at": "2026-05-01T12:00:00",
  "updated_at": "2026-05-01T12:00:00"
}
```

11.2. Пример запроса анализа влияния

```
POST /api/projects/{project_id}/impact

{
  "artifact_id": "uuid-of-REQ-001",
  "direction": "both",
  "max_depth": 3,
  "include_relation_types": ["depends_on", "realizes", "implemented_in", "verifies",
    "affects", "part_of"]
}
```

11.3. Пример ответа анализа влияния

```
{
  "source": {"code": "REQ-001", "title": "Пользователь должен авторизоваться"},
  "items": [
    {
      "artifact": {"code": "ENT-001", "type": "DomainConcept", "title": "Пользователь"},
      "depth": 1,
      "score": 0.85,
      "path": ["REQ-001", "implemented_in", "MOD-001", "depends_on", "ENT-001"]
    },
    {
      "artifact": {"code": "TEST-001", "type": "TestCase", "title": "Проверка авторизации"},
      "depth": 1,
      "score": 0.90,
      "path": ["TEST-001", "verifies", "REQ-001"]
    }
  ]
}
```

12. Алгоритмы и бизнес-правила

12.1. Алгоритм анализа влияния изменений

1. Получить выбранный исходный артефакт.
2. Загрузить все связи проекта или только связи выбранных типов.
3. Построить список смежности графа. Для `direction="forward"` учитывать исходящие связи, для `direction="backward"` — входящие, для `direction="both"` — оба направления.
4. Выполнить обход графа в ширину от исходного артефакта до `max_depth`.
5. Не добавлять один и тот же артефакт в результат повторно; если найден более короткий путь, обновить путь.
6. Для каждой найденной вершины сохранить глубину, путь и типы связей.
7. Рассчитать приблизительную оценку влияния: $\text{score} = \max(0.2, 1 - 0.25 * \text{depth}) * \text{relation_weight}$.
8. Сгруппировать результаты по типам артефактов.
9. Вернуть результат на frontend и подсветить найденные узлы на графе.

12.2. Проверка циклов

Для связей `part_of` и `is_a` перед сохранением нужно проверить, что добавление связи `source -> target` не создает путь `target -> source` по связям того же типа. Если такой путь существует, сохранение запрещается с сообщением: «Связь создает цикл в иерархии».

12.3. Диагностика неполной трассируемости

- Требование без связи с сущностью предметной области считается неполно описанным.
- Требование без связи с элементом реализации считается не реализованным или не привязанным к реализации.
- Требование без связи `verifies` со стороны теста считается непроверенным.
- Файл или функция без связи с требованием считается техническим элементом без явной трассируемости.
- Артефакт без входящих и исходящих связей считается изолированным.

13. Нефункциональные требования

ID	Категория	Требование
NFR-001	Производительность	Для проекта до 1000 артефактов и 3000 связей операции CRUD должны выполняться без заметной задержки; построение графа до 300 отображаемых узлов — до 2 секунд на обычном ноутбуке.
NFR-002	Простота развертывания	Проект должен запускаться локально через понятные команды backend и frontend. Желателен docker-compose, но он не обязателен для MVP.
NFR-003	Совместимость	Frontend должен работать в актуальных версиях Chrome, Edge и Firefox. Backend должен запускаться на Windows 10/11 и Linux.

ID	Категория	Требование
NFR-004	Поддерживаемость	Код должен быть разделен на слои: API, services, repositories/models, schemas, frontend pages/components/api-client.
NFR-005	Расширяемость	Добавление нового типа артефакта или связи не должно требовать переписывания архитектуры.
NFR-006	Надежность	Ошибки API должны возвращаться в едином формате. UI не должен падать при пустом проекте, отсутствии связей или ошибке валидации.
NFR-007	Безопасность MVP	Так как система локальная и однопользовательская, сложная авторизация не требуется. Однако API не должен выполнять произвольный код пользователя.
NFR-008	Удобство интерфейса	Основные действия должны быть доступны за 1-3 клика из соответствующего раздела. Ошибки должны быть понятны пользователю.
NFR-009	Резервное копирование	Экспорт JSON должен позволять сохранить проект и восстановить его через импорт.
NFR-010	Тестируемость	Ключевая бизнес-логика анализа влияния и проверки связей должна быть покрыта unit-тестами.

14. Требования к стеку технологий и архитектуре

Стек выбирается по принципу максимальной практичности: популярные инструменты, быстрый старт, хорошая документация, отсутствие тяжелой инфраструктуры. Для дипломного прототипа это важнее, чем промышленная сложность.

Часть	Технология	Обоснование
Frontend	React, TypeScript, Vite	Быстрая разработка SPA, типизация, простая сборка.
Граф	React Flow	Готовый инструмент для интерактивных графов: узлы, ребра, масштабирование, обработчики событий.
Редактор кода	Monaco Editor	Редактор, похожий на VS Code, с подсветкой синтаксиса.
Backend	Python, FastAPI	Python знаком пользователю, FastAPI быстро позволяет сделать REST API с OpenAPI-документацией.
ORM	SQLAlchemy	Удобная работа с БД и возможность заменить SQLite на PostgreSQL.
Валидация	Pydantic	Схемы запросов и ответов, валидация входных данных.
База данных	SQLite	Простая локальная БД без отдельного сервера, достаточная для прототипа.
Экспорт RDF	RDFLib	Формирование RDF/Turtle без отдельного RDF-сервера.
Тесты backend	pytest, FastAPI TestClient	Unit- и integration-тесты API и бизнес-логики.
Тесты frontend	Vitest/React Testing Library или ручное тестирование	Для MVP достаточно базовых smoke-тестов и ручных сценариев.

14.1. Предлагаемая структура репозитория

```
project-root/  
backend/  
  app/  
    main.py  
    api/  
      projects.py  
      artifacts.py  
      relations.py  
      graph.py  
      impact.py  
      export.py  
    core/  
      config.py  
      database.py  
    models/  
      project.py  
      artifact.py  
      relation.py  
      code_fragment.py  
    schemas/  
      project.py  
      artifact.py  
      relation.py  
      graph.py  
      impact.py  
    services/  
      artifact_service.py  
      relation_service.py  
      graph_service.py  
      impact_service.py  
      export_service.py  
      demo_data_service.py  
    tests/  
      test_impact.py  
      test_relations.py  
  requirements.txt  
frontend/  
  src/  
    app/  
    pages/  
      ProjectsPage.tsx  
      DashboardPage.tsx  
      ArtifactsPage.tsx  
      RequirementsPage.tsx  
      GraphPage.tsx  
      CodePage.tsx  
      ImpactPage.tsx  
    components/  
      ArtifactForm.tsx  
      RelationForm.tsx  
      GraphCanvas.tsx  
      CodeEditor.tsx  
      ImpactResult.tsx  
    api/  
      client.ts  
      projects.ts  
      artifacts.ts  
      relations.ts  
    types/  
      artifact.ts
```

```
graph.ts
main.tsx
package.json
docs/
  technical-specification.docx
  demo-scenario.md
README.md
```

14.2. Разделение ответственности backend

- api — маршруты HTTP, прием и возврат данных;
- schemas — Pydantic-схемы входных и выходных данных;
- models — SQLAlchemy-модели таблиц;
- services — бизнес-логика, проверки, анализ графа, экспорт;
- core — конфигурация, подключение к БД, общие настройки;
- tests — автоматические тесты бизнес-логики и API.

14.3. Разделение ответственности frontend

- pages — верхнеуровневые страницы приложения;
- components — переиспользуемые UI-компоненты;
- api — функции обращения к backend;
- types — TypeScript-типы сущностей API;
- GraphCanvas — компонент визуализации графа;
- CodeEditor — обертка над Monaco Editor;
- ImpactResult — компонент вывода результатов анализа влияния.

15. Развертывание и эксплуатация

15.1. Минимальные требования к окружению

Компонент	Требование
ОС	Windows 10/11 или Linux
Python	3.11 или новее
Node.js	20 LTS или новее
RAM	От 4 ГБ, рекомендуется 8 ГБ
Диск	От 500 МБ для проекта и зависимостей
Браузер	Chrome/Edge/Firefox актуальной версии

15.2. Команды запуска в режиме разработки

```
# Backend
cd backend
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate  # Linux
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend
cd frontend
npm install
npm run dev
```

15.3. Первичная настройка

1. При первом запуске backend создает SQLite-файл базы данных, если он отсутствует.
2. Система заполняет справочники artifact_types и relation_types стартовыми значениями.
3. Пользователь открывает frontend в браузере.
4. Пользователь создает проект или загружает демонстрационный проект.
5. После завершения работы пользователь может экспортировать проект в JSON.

16. Тестирование и приемка

Тестирование должно подтвердить, что система выполняет заявленные функции MVP: хранение артефактов, создание связей, построение графа, анализ влияния и экспорт модели.

16.1. Виды тестирования

- Unit-тесты backend: проверка алгоритмов анализа влияния, проверки циклов, RDF-экспорта.
- Integration-тесты API: создание проекта, артефактов, связей, получение графа.
- Ручное UI-тестирование: прохождение основных пользовательских сценариев.
- Приемочное тестирование: демонстрационный сценарий на подготовленном проекте.
- Регрессионная проверка: после изменения модели убедиться, что старые сценарии не сломались.

16.2. Тестовые сценарии

ID	Проверка	Ожидаемый результат
TC-001	Создать проект	Проект появляется в списке, открывается панель проекта.
TC-002	Создать требование	Требование сохраняется, получает код, отображается в таблице требований.
TC-003	Создать сущность предметной области	Сущность сохраняется и отображается в общем списке артефактов.
TC-004	Создать связь Requirement -> DomainConcept	Связь сохраняется и появляется в карточке артефакта.
TC-005	Запретить связь с самим собой	Система показывает ошибку и не сохраняет связь.
TC-006	Запретить цикл part_of	Система показывает ошибку при попытке создать цикл.
TC-007	Построить граф	Граф отображает созданные узлы и ребра.
TC-008	Запустить анализ влияния	Система возвращает связанные элементы и пути влияния.
TC-009	Создать фрагмент кода	Фрагмент сохраняется и отображается в редакторе.
TC-010	Экспортировать JSON	Файл содержит проект, артефакты и связи.
TC-011	Экспортировать RDF/Turtle	Файл содержит RDF-тройки для артефактов и связей.
TC-012	Загрузить демо-проект	В системе появляется заранее подготовленный проект с требованиями, графом и кодом.

16.3. Критерии приемки MVP

1. Приложение запускается локально по инструкции.
2. Можно создать проект и не менее пяти типов артефактов.
3. Можно создать не менее пяти типов связей между артефактами.
4. Граф проекта отображает артефакты и связи.
5. Анализ влияния для выбранного требования возвращает связанные сущности, элементы реализации и тесты.
6. Можно добавить и просмотреть фрагмент кода.
7. Экспорт JSON и RDF/Turtle работает без ошибок.
8. Демонстрационный проект загружается и позволяет показать все ключевые функции.
9. Основные сценарии TC-001 — TC-012 пройдены.
10. Кодовая база имеет понятную структуру, позволяющую описать архитектуру в отчете.

17. Демонстрационный сценарий

Демонстрационный проект нужен для защиты и для быстрой проверки работоспособности. Рекомендуемый пример — небольшая система управления задачами или авторизации, потому что она понятна проверяющим и легко связывается с требованиями, сущностями, кодом и тестами.

17.1. Демонстрационные данные

Код	Тип	Название
ENT-001	DomainConcept	Пользователь
ENT-002	DomainConcept	Учетная запись
ENT-003	DomainConcept	Сессия
REQ-001	Requirement	Пользователь должен авторизоваться
REQ-002	Requirement	Система должна хранить активную сессию
MOD-001	Module	Auth
FILE-001	File	auth.py
FUNC-001	Function	login_user
TEST-001	TestCase	Проверка успешной авторизации
DOC-001	Document	Описание механизма авторизации

17.2. Демонстрационные связи

Источник	Связь	Цель
REQ-001	depends_on	ENT-001
REQ-001	depends_on	ENT-002
REQ-002	depends_on	ENT-003
MOD-001	realizes	REQ-001
FILE-001	part_of	MOD-001
FUNC-001	part_of	FILE-001
FUNC-001	realizes	REQ-001
TEST-001	verifies	REQ-001

Источник	Связь	Цель
DOC-001	describes	REQ-001

17.3. Что показать на защите

1. Открыть проект и показать панель со статистикой.
2. Открыть требования и показать REQ-001.
3. Показать связанные сущности предметной области.
4. Открыть граф и выделить REQ-001.
5. Запустить анализ влияния для REQ-001.
6. Показать, что затронуты Auth, auth.py, login_user, тест и документация.
7. Открыть фрагмент кода функции login_user.
8. Экспортировать RDF/Turtle и показать, что модель представлена в виде семантических троек.

18. План реализации

Этап	Название	Результат
Этап 1	Подготовка проекта	Создать репозиторий, backend, frontend, базовую структуру, README.
Этап 2	База данных и справочники	Создать модели SQLAlchemy, SQLite, начальные artifact_types и relation_types.
Этап 3	CRUD проектов и артефактов	Реализовать API и страницы списка/карточки артефакта.
Этап 4	Связи и проверки	Реализовать создание связей, проверки типов и циклов.
Этап 5	Граф	Подключить React Flow и вывести nodes/edges.
Этап 6	Код	Подключить Monaco Editor и хранение code_fragments.
Этап 7	Анализ влияния	Реализовать BFS-алгоритм и экран результата.
Этап 8	Экспорт	Реализовать JSON и RDF/Turtle.
Этап 9	Демо-данные и тесты	Добавить demo_data_service и тестовые сценарии.
Этап 10	Полировка	Проверить UI, исправить ошибки, подготовить скриншоты и описание для отчета.

19. Риски и решения

Риск	Вероятность	Мера снижения
Слишком широкая тема	Высокая	Жестко держаться MVP: артефакты, связи, граф, анализ влияния, экспорт. Не делать полноценную IDE.
Сложность онтологий	Высокая	Использовать прикладную онтологическую модель и RDF-экспорт; OWL reasoner оставить как перспективу.
Сложность графа	Средняя	Использовать React Flow, ограничить количество отображаемых узлов, добавить фильтры.

Риск	Вероятность	Мера снижения
Нехватка времени	Высокая	Сначала реализовать backend CRUD и простую таблицу, затем граф, затем анализ влияния.
Неубедительность программы	Средняя	Подготовить демонстрационный проект с понятным сценарием и связями требования-код-тест.
Ошибки в данных	Средняя	Ввести проверки обязательных полей, уникальности кодов и циклов.
Слишком сложный стек	Средняя	Не поднимать отдельный RDF-сервер и PostgreSQL в MVP; использовать SQLite и RDFLib.

Приложение А. Минимальный справочник типов артефактов

Код	Название	Категория
DomainConcept	Понятие предметной области	domain
Requirement	Требование	requirements
Module	Модуль	implementation
Component	Компонент	implementation
File	Файл	implementation
Function	Функция/метод/endpoint	implementation
TestCase	Тестовый сценарий	testing
Document	Документ	documentation
Defect	Дефект	quality
ChangeRequest	Запрос на изменение	management
Decision	Проектное решение	architecture

Приложение Б. Минимальный справочник типов связей

Код	Название	Направленная	Участвует в анализе
is_a	Род-вид / специализация	Да	Нет
part_of	Часть-целое	Да	Да
depends_on	Зависит от	Да	Да
realizes	Реализует	Да	Да
implemented_in	Реализовано в	Да	Да
describes	Описывает	Да	Нет
verifies	Проверяет	Да	Да
affects	Влияет на	Да	Да
conflicts_with	Конфликтует с	Нет	Нет
related_to	Связан с	Нет	Нет
refines	Уточняет	Да	Да
uses	Использует	Да	Да

Приложение В. Псевдокод анализа влияния

```
def analyze_impact(project_id, source_id, max_depth=3, direction="both",
relation_types=None):
    relations = load_relations(project_id, relation_types)
    graph = build_adjacency(relations, direction)

    queue = [(source_id, 0, [])]
    visited = {source_id}
```

```

result = []

while queue:
    current_id, depth, path = queue.pop(0)
    if depth >= max_depth:
        continue

    for edge in graph[current_id]:
        next_id = edge.target_id
        if next_id in visited:
            continue

        visited.add(next_id)
        next_path = path + [edge]
        score = max(0.2, 1 - 0.25 * (depth + 1)) * edge.weight

        result.append({
            "artifact_id": next_id,
            "depth": depth + 1,
            "score": score,
            "path": next_path,
        })

        queue.append((next_id, depth + 1, next_path))

return group_by_artifact_type(result)

```

Приложение Г. Минимальный README для разработчика

В репозитории должен быть README.md, в котором описаны: назначение проекта, стек, структура папок, запуск backend, запуск frontend, создание/загрузка демо-проекта, запуск тестов, известные ограничения MVP.

Workplace Lifecycle Interface

Прототип веб-интерфейса для управления связанными артефактами рабочего места программиста.

Возможности

- проекты и артефакты;
- требования и сущности предметной области;
- связи между артефактами;
- граф проекта;
- анализ влияния изменений;
- фрагменты кода;
- экспорт JSON и RDF/Turtle.

Запуск

1. Запустить backend: `uvicorn app.main:app --reload --port 8000`
2. Запустить frontend: `npm run dev`
3. Открыть приложение в браузере.

Демо

Нажать «Загрузить демонстрационный проект» и открыть граф.

Приложение Д. Что можно оставить на развитие после MVP

- Авторизация и роли пользователей.

- Интеграция с Git и автоматическое связывание commits/files/functions.
- Импорт требований из Markdown, CSV или DOCX.
- Расширенный OWL-экспорт и подключение reasoner.
- SPARQL-запросы к модели проекта.
- Автоматическое извлечение сущностей из описания предметной области.
- История версий артефактов и сравнение изменений.
- Генерация отчетов по трассируемости.
- Уведомления о неполной трассируемости и конфликтах.
- Многопользовательская работа и совместное редактирование.

Краткий итог

Данное техническое задание фиксирует реалистичную трактовку темы: не создание полноценной среды разработки, а разработку прототипа семантического навигатора по проектным артефактам программиста. Минимально необходимый результат — приложение, в котором можно завести проект, требования, сущности предметной области, элементы реализации, связи между ними, увидеть граф и получить список элементов, потенциально затронутых изменением требования или другой сущности.